

Optical Fingerprint Sensor AS608

Hardware Overview

What is it?

The **AS608** is a biometric optical scanner. It has an internal Digital Signal Processor (DSP) that analyzes a fingerprint, converts that geometry into a 512-byte mathematical model, and stores it in its own onboard flash memory.

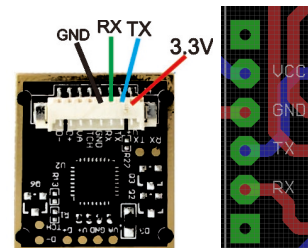
Electrical Explanation

The module operates using Serial communication. When instructed, a bright LED illuminates the glass prism. The internal processor captures the image, mathematically maps the minutiae (ridge endings and bifurcations), and compares it against its internal database.

- **The Power Supply:** The raw silicon chip operates strictly at 3.3V meaning if you plug VCC into a 5V source, the chip will instantly fry.
- **Current Draw:** When the LED flashes and the onboard processor crunches the math, the sensor pulls up to ~120mA.

Key Hardware Facts:

- **TX Pin:** The K-Comp sends commands on the 3.3V RX Pin of the 3.3V Expansion Slot
- **RX Pin:** The K-Comp receives data on the TX Pin of the 3.3V Expansion Slot
- **Baud Rate:** Usually defaults to 57600.
- **Voltage:** Operates at 3.3V. **Only use the 3.3V Pins!**
- **Signal Flow:** LED Flash → Image Capture → DSP Math → Serial Connection → K-Comp.



kcomp_AS608.h

This library is a "Bare-Metal Flat C" parser. It manually calculates and unpacks the raw hexadecimal packets traveling across the wire.

Usage: `#include <kcomp_AS608.h>`

Function Reference

Function Name	Description & Parameters
<code>bool initAS608(rxPin, txPin, baud)</code>	Setup: Initializes the Fingerprint Reader. ▪ rx/tx: Pin 6 and Pin 7. ▪ baud: Speed (usually 57600). Returns: <code>true</code> if successful.
<code>int16_t scanFingerprint()</code>	Scan: Triggers the reader. Returns the matched User ID (0-127). Returns negative error codes if no match is found.

<code>int8_t enrollFingerprint(uint_16t id)</code>	Enroll: Blocking function. Prompts the user to place their finger twice to create a robust mathematical model, saving it to the specified id slot.
<code>int16_t getFingerprintCount()</code>	Queries the sensor to see exactly how many fingerprints are currently stored in memory.
<code>bool isSlotOccupied(uint_16t id)</code>	Checks if a specific ID slot is already taken before you accidentally overwrite it.
<code>int8_t deleteFingerprint(uint_16t id)</code>	Deletes a single fingerprint ID from the memory.
<code>bool clearAllFingerprints()</code>	Performs a factory reset, instantly wiping all stored fingerprints.
<code>void controlled(uint_8t mode, uint_16t color, uint_16t speed, uint_16t cycles)</code>	Hardware: Controls the LED ring (on R503 variants). Modes include <code>LED_BREATHING</code> , <code>LED_FLASHING</code> , <code>LED_ALWAYS_ON</code> . Colors include <code>LED_RED</code> , <code>LED_BLUE</code> , <code>LED_PURPLE</code> .

⚠ Pitfalls & Troubleshooting

"Sensor not found!"

- Serial communication is crossed. The K-Comp's RX must connect to the sensor's TX and vice versa. Only connect to 3.3V!

Can I download the actual picture of the fingerprint?

An 8-bit grayscale image is ~36KB. K-Comp only has 2KB of RAM. It will instantly crash.

The Fix: You cannot use standard variables for this. You must write a specific "code" sketch that immediately passes incoming serial bytes directly to the PC without storing them.

Sample Program

This program turns the Arduino Serial Monitor into an interactive command console, allowing students to manage the database and scan fingers.

```
#include <kcomp_AS608.h>

#define RX_PIN 6    // Connect to Sensor TX
#define TX_PIN 7    // Connect to Sensor RX
#define BAUD 57600 // Change to 9600 if it fails to connect!

void setup() {
  Serial.begin(115200);
  while (!Serial);

  Serial.println("Connecting to Sensor...");

  if (initAS608(RX_PIN, TX_PIN, BAUD)) {
    Serial.println("[SUCCESS] Sensor Ready!");
  } else {
    Serial.println("[ERROR] Sensor not found. Check RX/TX crossover.");
    while (1); // Halt
  }
}

void loop() {
  Serial.println("\n--- MAIN MENU ---");
```

```
Serial.println("1: Scan Finger");
Serial.println("2: Enroll New User");
Serial.println("3: Check Capacity");
Serial.println("Type your choice and press Enter:");

// Wait for user input
while (!Serial.available());
int choice = Serial.parseInt();

// Clean up the 'Enter' key press from the buffer
delay(10);
while (Serial.available()) Serial.read();

if (choice == 1) {
    Serial.println("\n[SCAN MODE] Place finger on glass...");
    int16_t id = -1;
    while (id == -1) { id = scanFingerprint(); } // Wait for scan

    if (id >= 0) {
        Serial.print("ACCESS GRANTED! ID: ");
        Serial.println(id);
    } else {
        Serial.println("ACCESS DENIED.");
    }
}
else if (choice == 2) {
    Serial.println("\n[ENROLL MODE] Enter a User ID (1-127):");
    while (!Serial.available());
    int target = Serial.parseInt();

    if (enrollFingerprint(target) == AS608_OK) {
        Serial.println("Successfully Enrolled!");
    } else {
        Serial.println("Enrollment Failed.");
    }
}
else if (choice == 3) {
    Serial.print("\nTotal Users Stored: ");
    Serial.println(getFingerprintCount());
}

delay(2000); // Give user time to read output before showing menu again
}
```

💡 **Try it out:** You can find the full sample program **SimpleFingerprintReaderAS608** in the K-Comp examples folder!